フロントエンド実装設計書

Loggit

Version 1.0　　作成日：2026 年 2 月 24 日

1. 技術スタック

カテゴリ	採用技術
フレームワーク	React 18（将来的に Next.js へ移行検討）
言語	TypeScript
ビルドツール	Vite
スタイリング	Tailwind CSS
HTTP クライアント	axios
ルーティング	React Router v6
状態管理	React Context + useState（シンプル構成）
フォーム	React Hook Form
日付処理	Day.js
グラフ	Recharts

2. ディレクトリ構成

```
frontend/
├── public/
├── src/
│   ├── api/                    # axios クライアント・API リクエスト関数
│   │   ├── client.ts           # axios インスタンス・インターセプター設定
│   │   ├── auth.ts             # 認証系 API リクエスト
│   │   ├── projects.ts         # プロジェクト系 API リクエスト
│   │   ├── tasks.ts            # タスク系 API リクエスト
│   │   ├── activityLogs.ts     # 活動ログ系 API リクエスト
│   │   ├── timeLogs.ts         # 時間配賦系 API リクエスト
│   │   └── settings.ts         # 設定系 API リクエスト
│   │
│   ├── components/             # 再利用可能な UI コンポーネント
│   │   ├── common/             # 汎用コンポーネント
│   │   │   ├── Button.tsx
│   │   │   ├── Modal.tsx
│   │   │   ├── Toast.tsx
│   │   │   ├── Skeleton.tsx
│   │   │   └── EmptyState.tsx
```

```
│   │   ├── layout/            # レイアウト系
│   │   │   ├── Sidebar.tsx
│   │   │   ├── Topbar.tsx
│   │   │   └── AppLayout.tsx  # Sidebar+Topbar+children を組み合わせ
│   │   ├── tracker/           # 打刻ウィジェット
│   │   │   ├── TrackerWidget.tsx
│   │   │   └── TrackerTimer.tsx
│   │   ├── project/           # プロジェクト関連
│   │   │   ├── ProjectCard.tsx
│   │   │   ├── ProjectForm.tsx
│   │   │   └── StatusColumn.tsx
│   │   ├── task/              # タスク関連
│   │   │   ├── TaskItem.tsx
│   │   │   ├── TaskSidePanel.tsx
│   │   │   └── TaskForm.tsx
│   │   ├── activityLog/       # 活動ログ関連
│   │   │   ├── LogItem.tsx
│   │   │   └── AllocateModal.tsx  # 配賦モーダル
│   │   └── dashboard/         # ダッシュボード専用
│   │       ├── StatCard.tsx
│   │       ├── WeeklyChart.tsx
│   │       └── ProjectProgress.tsx
│   │
│   ├── pages/                 # ページコンポーネント（ルートに対応）
│   │   ├── Dashboard.tsx      # /
│   │   ├── ProjectList.tsx    # /projects
│   │   ├── ProjectDetail.tsx  # /projects/:id
│   │   ├── TaskList.tsx       # /tasks
│   │   ├── ActivityLog.tsx    # /activity-logs
│   │   ├── Settings.tsx       # /settings
│   │   ├── Login.tsx          # /login
│   │   └── Register.tsx       # /register
│   │
│   ├── hooks/                 # カスタムフック
│   │   ├── useTracker.ts      # 打刻状態管理・タイマー
│   │   ├── useToast.ts        # トースト通知
│   │   ├── useAuth.ts         # 認証状態
│   │   └── useUrlQuery.ts     # URL クエリパラメータ同期
│   │
│   ├── contexts/              # React Context
│   │   ├── AuthContext.tsx    # 認証状態・ユーザー情報
│   │   └── ToastContext.tsx   # トースト通知
│   │
│   ├── types/                 # TypeScript 型定義
│   │   ├── project.ts
│   │   ├── task.ts
│   │   ├── activityLog.ts
│   │   ├── timeLog.ts
│   │   └── user.ts
│   │
│   ├── utils/                 # ユーティリティ関数
│   │   ├── formatTime.ts      # 分→「Xh Ym」変換
│   │   ├── formatDate.ts      # 日付フォーマット
│   │   └── priorityLabel.ts   # priority 数値→ラベル変換
│   │
│   ├── App.tsx                # ルーティング定義
│   └── main.tsx               # エントリーポイント
│
├── index.html
├── vite.config.ts
├── tailwind.config.ts
└── tsconfig.json
```

3. ルーティング定義（App.tsx）

```
// src/App.tsx
const App = () => (
  <AuthProvider>
    <ToastProvider>
      <Router>
        <Routes>
          {/* 未認証 */}
          <Route path='/login'    element={<Login />} />
          <Route path='/register' element={<Register />} />

          {/* 認証必須（PrivateRoute でラップ） */}
          <Route element={<PrivateRoute />}>
            <Route element={<AppLayout />}>
              <Route path='/'                 element={<Dashboard />} />
              <Route path='/projects'         element={<ProjectList />} />
              <Route path='/projects/:id'     element={<ProjectDetail />} />
              <Route path='/tasks'            element={<TaskList />} />
              <Route path='/activity-logs'    element={<ActivityLog />} />
              <Route path='/settings'         element={<Settings />} />
            </Route>
          </Route>
        </Routes>
      </Router>
    </ToastProvider>
  </AuthProvider>
);
```

4. API クライアント設計（api/client.ts）

```
// src/api/client.ts
import axios from 'axios';

const client = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,  // .env で設定
  headers: {
    'Content-Type': 'application/json',
    'Accept':        'application/json',
  },
});

// リクエストインターセプター: Bearer トークンを自動付与
client.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// レスポンスインターセプター: 401 検知→ログイン画面へ
client.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(err);
  }
);

export default client;
```

5. TypeScript 型定義（types/）

5.1 user.ts

```
export type User = {
  id: number;
  name: string;
  email: string;
  created_at: string;
};

export type UserSetting = {
  enable_task_time_tracking: boolean;
};
```

5.2 project.ts

```
export type ProjectStatus = {
  id: number;
  name: string;
  order: number;
};

export type Project = {
  id: number;
  name: string;
  description: string | null;
  color: string | null;
  due_date: string | null;
  task_count: number;
  completed_task_count: number;
  statuses: ProjectStatus[];
  created_at: string;
  updated_at: string;
};
```

5.3 task.ts

```
export type Priority = 1 | 2 | 3;
export type PriorityLabel = 'low' | 'mid' | 'high';

export type Task = {
  id: number;
  project_id: number;
  project_name: string;
  parent_task_id: number | null;
  status: ProjectStatus;
  title: string;
  description: string | null;
  priority: Priority;
  priority_label: PriorityLabel;
  due_date: string | null;
  subtasks: Task[];
  subtask_count: number;
  completed_subtask_count: number;
  total_tracked_minutes: number;
```

```
  deleted_at: string | null;
  created_at: string;
  updated_at: string;
};
```

5.4 activityLog.ts

```
export type ActivityLog = {
  id: number;
  project_id: number | null;
  project_name: string | null;
  started_at: string;
  ended_at: string | null;
  duration_minutes: number | null;
  note: string | null;
  auto_stopped: boolean;
  is_tracking: boolean;
  task_time_logs: TaskTimeLog[];
};
```

5.5 timeLog.ts

```
export type TaskTimeLog = {
  id: number;
  task_id: number;
  activity_log_id: number | null;
  activity_started_at: string | null;
  activity_duration_minutes: number | null;
  duration_minutes: number;
  worked_on: string | null;
  note: string | null;
  created_at: string;
};
```

6. Context 設計

6.1 AuthContext

```
// src/contexts/AuthContext.tsx
type AuthContextType = {
  user: User | null;
  token: string | null;
  login: (email: string, password: string) => Promise<void>;
  logout: () => Promise<void>;
  isAuthenticated: boolean;
};

// 提供する state
// - user: ログイン中のユーザー情報
// - token: Sanctum トークン（localStorage に保存）
// - isAuthenticated: 認証済みかどうか
```

6.2 ToastContext

```
// src/contexts/ToastContext.tsx
```

```
type ToastType = 'success' | 'error' | 'warning';

type ToastContextType = {
  showToast: (message: string, type: ToastType) => void;
};

// 使用例
// const { showToast } = useToast();
// showToast('保存しました', 'success');  → 緑・3秒で自動消去
// showToast('エラーが発生しました', 'error'); → 赤・3秒で自動消去
```

7. カスタムフック定義

7.1 useTracker（打刻状態管理）

```
// src/hooks/useTracker.ts
const useTracker = () => {
  const [activeLog, setActiveLog] = useState<ActivityLog | null>(null);
  const [elapsed, setElapsed]     = useState<number>(0); // 経過秒数

  // ページ読み込み時に計測中ログを取得
  useEffect(() => {
    fetchActivityLogs().then((logs) => {
      const tracking = logs.find((l) => l.is_tracking);
      if (tracking) setActiveLog(tracking);
    });
  }, []);

  // 1秒ごとにelapsedをインクリメント
  useEffect(() => {
    if (!activeLog) return;
    const timer = setInterval(() => setElapsed((e) => e + 1), 1000);
    return () => clearInterval(timer);
  }, [activeLog]);

  const start = async (projectId?: number, note?: string) => { ... };
  const stop  = async () => { ... }; // 停止後に配賦モーダルを表示

  return { activeLog, elapsed, start, stop };
};
```

7.2 useUrlQuery（URL クエリ同期）

```
// src/hooks/useUrlQuery.ts
const useUrlQuery = <T extends Record<string, string>>(defaults: T) => {
  const [searchParams, setSearchParams] = useSearchParams();

  const query = useMemo(() => ({
    ...defaults,
    ...Object.fromEntries(searchParams.entries()),
  }), [searchParams]);

  // 300msデバウンスでURLを更新
  const setQuery = useDebouncedCallback((updates: Partial<T>) => {
    setSearchParams({ ...query, ...updates });
  }, 300);

  return { query, setQuery };
```

```
};

// 使用例（TaskList.tsx）
// const { query, setQuery } = useUrlQuery({
//   sort: 'created_at_desc',
//   project_id: '',
//   status_id: '',
// });
```

7.3 useAuth

```
// src/hooks/useAuth.ts
const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth must be used within AuthProvider');
  return context;
};
```

8. 主要コンポーネント責務

8.1 AppLayout

- Sidebar・Topbar・メインコンテンツエリアを組み合わせた共通レイアウト
- TrackerWidget を Topbar 内に常時表示
- Outlet（React Router）でページコンポーネントをレンダリング

8.2 TrackerWidget

- useTracker フックから打刻状態・経過時間を取得して表示
- 計測中：タイマー表示・停止ボタン
- 停止中：開始ボタン・プロジェクト選択
- 停止ボタン押下後に AllocateModal を表示

8.3 AllocateModal（配賦モーダル）

- 停止後に自動表示。スキップ可能
- Props：activityLog（停止したログ）・onClose
- タスク選択（プロジェクト→タスクの 2 段階）＋分単位入力
- 「＋ タスクを追加」ボタンで行を追加
- 合計配賦時間をリアルタイム計算・総活動時間超過で赤色警告
- 保存ボタン押下で POST /api/tasks/{id}/time-logs を複数回コール

8.4 TaskSidePanel

- タスク行クリックで右からスライドイン
- Props：taskId・onClose
- 表示時に GET /api/tasks/{id}と GET /api/tasks/{id}/time-logs を並列取得
- タイトル・ステータス・優先度・期限をインライン編集可能

- 累計時間・配賦一覧を表示
- 「時間を手動追加」ボタンで手動配賦フォームを表示
- 削除済みサブタスクは「削除済みタスク」グレー表示＋復元ボタン

8.5 StatusColumn（プロジェクット詳細）

- Jira ライクなカンバンボードの 1 カラム
- Props：status・tasks・onTaskClick
- ステータス名をクリックでインライン編集可能
- タスクカードを縦に並べて表示

9. 状態管理方針

グローバルな状態管理ライブラリ（Redux・Zustand 等）は使用しない。シンプルな構成を維持する。

状態の種類	管理方法
認証状態・ユーザー情報	AuthContext（全画面で参照）
トースト通知	ToastContext（全画面で参照）
打刻状態・タイマー	useTracker フック（AppLayout で保持）
一覧データ（タスク・ログ等）	各ページの useState（ローカル）
フィルター・ソート状態	URL クエリパラメータ（useUrlQuery）
フォーム状態	React Hook Form（各フォームコンポーネント内）
モーダル表示状態	親コンポーネントの useState

10. API リクエスト関数（api/）

10.1 activityLogs.ts

```
// src/api/activityLogs.ts
import client from './client';
import type { ActivityLog } from '@/types/activityLog';

export const fetchActivityLogs = async (params?: Record<string, string>) => {
  const res = await client.get('/activity-logs', { params });
  return res.data;
};

export const startTracking = async (data: {
  project_id?: number;
  note?: string;
}): Promise<ActivityLog> => {
  const res = await client.post('/activity-logs/start', data);
  return res.data.data;
};
```

```
export const stopTracking = async (id: number): Promise<ActivityLog> => {
  const res = await client.post(`/activity-logs/${id}/stop`);
  return res.data.data;
};

export const fetchSummary = async (period: 'week' | 'month') => {
  const res = await client.get('/activity-logs/summary', { params: { period } });
  return res.data.data;
};
```

10.2 tasks.ts

```
// src/api/tasks.ts
import client from './client';
import type { Task } from '@/types/task';

export const fetchTasks = async (params?: Record<string, string>) => {
  const res = await client.get('/tasks', { params });
  return res.data;
};

export const fetchTask = async (id: number): Promise<Task> => {
  const res = await client.get(`/tasks/${id}`);
  return res.data.data;
};

export const createTask = async (projectId: number, data: Partial<Task>) => {
  const res = await client.post(`/projects/${projectId}/tasks`, data);
  return res.data.data;
};

export const updateTask = async (id: number, data: Partial<Task>) => {
  const res = await client.put(`/tasks/${id}`, data);
  return res.data.data;
};

export const deleteTask = async (id: number) => {
  await client.delete(`/tasks/${id}`);
};

export const restoreTask = async (id: number): Promise<Task> => {
  const res = await client.post(`/tasks/${id}/restore`);
  return res.data.data;
};
```

11. ユーティリティ関数（utils/）

11.1 formatTime.ts

```
// src/utils/formatTime.ts
export const formatMinutes = (minutes: number): string => {
  const h = Math.floor(minutes / 60);
  const m = minutes % 60;
  if (h === 0) return `${m}m`;
  if (m === 0) return `${h}h`;
  return `${h}h ${m}m`;
};
```

```
// 経過秒数 → HH:MM:SS
export const formatElapsed = (seconds: number): string => {
  const h = Math.floor(seconds / 3600).toString().padStart(2, '0');
  const m = Math.floor((seconds % 3600) / 60).toString().padStart(2, '0');
  const s = (seconds % 60).toString().padStart(2, '0');
  return `${h}:${m}:${s}`;
};
```

11.2 priorityLabel.ts

```
// src/utils/priorityLabel.ts
export const PRIORITY_LABELS = {
  1: { label: 'low',  color: 'gray' },
  2: { label: 'mid',  color: 'orange' },
  3: { label: 'high', color: 'red' },
} as const;

export const getPriorityLabel = (priority: 1 | 2 | 3) => {
  return PRIORITY_LABELS[priority];
};
```

12. 画面別コンポーネント構成

12.1 ダッシュボード（Dashboard.tsx）

コンポーネント	役割
StatCard × 3	今週の活動時間・配賦時間・今日の期限タスク数
WeeklyChart	週次・月次バーチャート（Recharts）
TaskItem × N（今日の期限）	期限タスク一覧・完了チェック
ProjectProgress × N	プロジェクト別進捗バー
LogItem × N（直近ログ）	活動ログ履歴・計測中表示

12.2 プロジェクト詳細（ProjectDetail.tsx）

コンポーネント	役割
StatusColumn × 3	カンバンボード（ステータス別カラム）
TaskItem × N	各カラム内のタスクカード
TaskSidePanel	タスク詳細・編集・累計時間表示
TaskForm	タスク新規作成フォーム（モーダル）

12.3 タスク横断一覧（TaskList.tsx）

コンポーネント	役割
フィルターバー	プロジェクト・ステータス・優先度・ソート（URL クエリ同期）
TaskItem × N	タスク行・クリックでサイドパネル表示
TaskSidePanel	タスク詳細・編集
Skeleton	ローディング中のスケルトン UI
EmptyState	タスクが 0 件の場合の空状態

12.4 活動ログ（ActivityLog.tsx）

コンポーネント	役割
フィルターバー	プロジェクト・期間絞り込み
LogItem × N	ログ行・配賦する/編集ボタン
AllocateModal	配賦モーダル（後から編集も可）
自動停止通知バナー	auto_stopped=true のログ検出時に表示

13. 環境変数（.env）

```
# frontend/.env
VITE_API_BASE_URL=http://localhost:8000/api

# frontend/.env.production
VITE_API_BASE_URL=https://api.loggit.example.com/api
```

14. 推奨実装順序

順番	内容	詳細
1	環境構築	Vite + React + TypeScript + Tailwind CSS セットアップ
2	API クライアント	axios 設定・インターセプター・型定義
3	認証画面	ログイン・登録・AuthContext・PrivateRoute
4	共通レイアウト	Sidebar・Topbar・AppLayout・ToastContext
5	打刻ウィジェット	TrackerWidget・useTracker・タイマー
6	プロジェクト管理	一覧・詳細・カンバンボード・ステータス編集
7	タスク管理	横断一覧・サイドパネル・フィルター・URL クエリ同期

順番	内容	詳細
8	活動ログ	一覧・配賦モーダル・自動停止通知バナー
9	ダッシュボード	集計・グラフ・統計カード
10	設定画面	プロフィール・パスワード・トラッキング ON/OFF